PROGRAMMING FUNDAMENTALS

Spring 2024

9th May

Getting the Address of a Variable

- It is done with an Amersand operator in C++
- when & is placed in front of a variable name, it returns the address of that variable
- cout<< &amount will display the variable's address on the screen

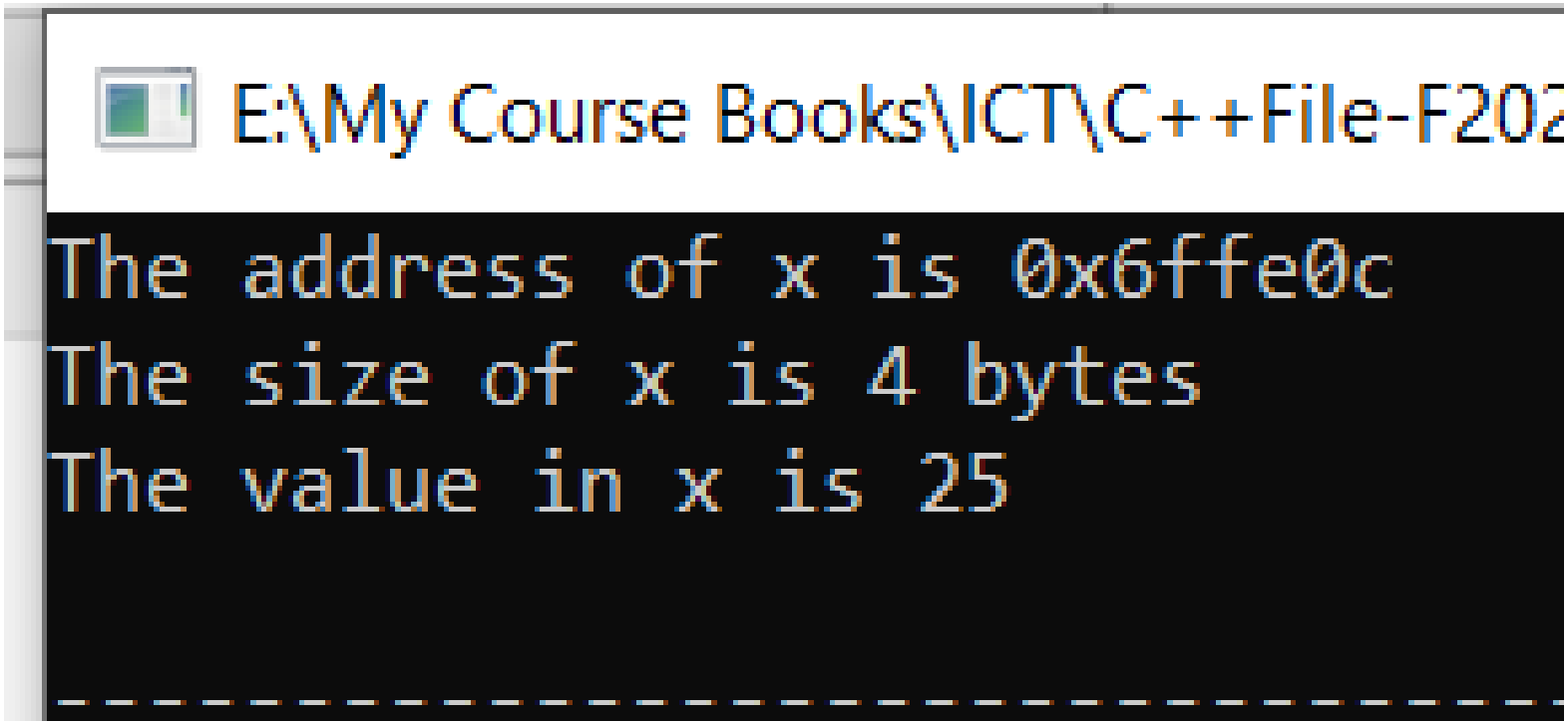
Ex-1

20-Lec.cpp

```cpp
#include <iostream>
using namespace std;

int main()
 {
    int x = 25;

    cout << "The address of x is " << &x << endl;
    cout << "The size of x is " << sizeof(x) << " bytes\n";
    cout << "The value in x is " << x << endl;
    return 0;
}
```


E:\My Course Books\ICT\C++File-F20
The address of x is 0x6ffe0c
The size of x is 4 bytes
The value in x is 25

Pre defined Function

- The C++ sizeof operator is **a unary operator that calculates the size of data type, variables, and constants at compile time.**
- The size returned from the operator is multiple of the size of char, which in most computers is 1 byte (8 bits).

Pointer Variables

- Pointer variables are designed to hold memory addresses
- it can be used to hold the location of some other piece of data
- It may point to some piece of data that is stored in the computer's memory
- Pointer variables also allow you to work with the data that they point to

Array passing to a function

```
const int SIZE = 5;
int numbers[SIZE] = { 1, 2, 3, 4, 5 };
showValues(numbers, SIZE);
```

- when we pass an array as an argument to a function, we are actually passing the array's beginning address

Function Definition

```
void showValues(int values[], int size)
{
for (int i = 0; i < size; i++)
cout << values[i] << endl;
}
```

- the values parameter receives the address of the numbers array
- It works like a pointer because it “points” to the numbers array

Ex-2

- Function call statement

```
int Item;
getOrder(Item);

Function Definition
void getOrder(int &Books)
{
cout << "How many Books do you want? ";
cin >> Books;
}
```

What is going on…

- the Books parameter is a reference variable, and it receives the address of the Item variable
- It works like a pointer because it “points” to the Item variable
- Inside the getOrder function, the Books parameter references the Item variable
- Anything done to the Books parameter is actually done to the Item variable
- The cin statement uses the Books reference variable to indirectly store the value in the Item variable

Now we have pointers..

- Pointer variables are similar to reference variables
- BUT
- A pointer variable references another item in memory
- You have to write code that fetches the memory address of that item and assigns the address to the pointer variable

Pointers

- Pointers are useful for dynamic memory allocation
- When you are writing a program that will need to work with an unknown amount of data, dynamic memory allocation allows you to create variables, arrays, etc while the program is running

Lets look at Pointer again

int *ptr;

- the word int does not mean ptr is an integer variable
- It means ptr can hold the address of an integer variable
- Pointers only hold one kind of value: an address

Initializing Pointer variables

- Define a pointer variable and initialize it with a valid memory address
- If you don't initialize then you may be affecting some unknown location in memory
- It is a good idea to initialize pointer variables with the special value nullptr

Initializing Pointer variables

- The nullptr key word was introduced to represent the address 0
- So, assigning nullptr to a pointer variable makes the variable point to the address 0
- → it points to “nothing”

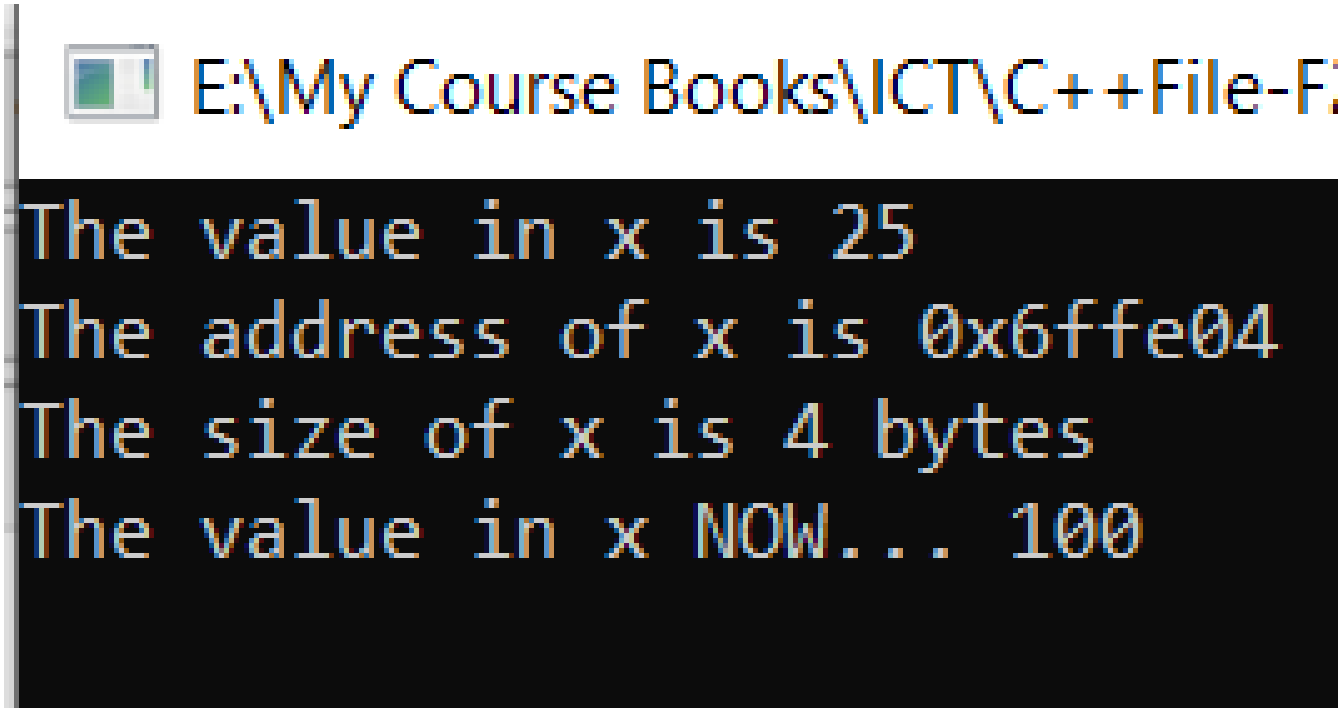
```
int *ptr = nullptr;
```

Or

```
int *ptr = 0;
```

Ex-3

```
int main()
 {
    int x = 25;
    int *ptr = 0;
    cout << "The value in x is " << x << endl;
    cout << "The address of x is " << &x << endl;
    cout << "The size of x is " << sizeof(x) << " bytes\n";
    ptr = &x;
    *ptr = 100;
    cout << "The value in x NOW... " << x << endl;
    return 0;
}
```


E:\My Course Books\ICT\C++File-F
The value in x is 25
The address of x is 0x6ffe04
The size of x is 4 bytes
The value in x NOW... 100

ptr = &x;

- This statement assigns the address of the x variable to the ptr variable.

- When you apply the indirection operator (*) to a pointer variable, you are working not with the pointer variable itself, but with the item it points to

*ptr = 100;

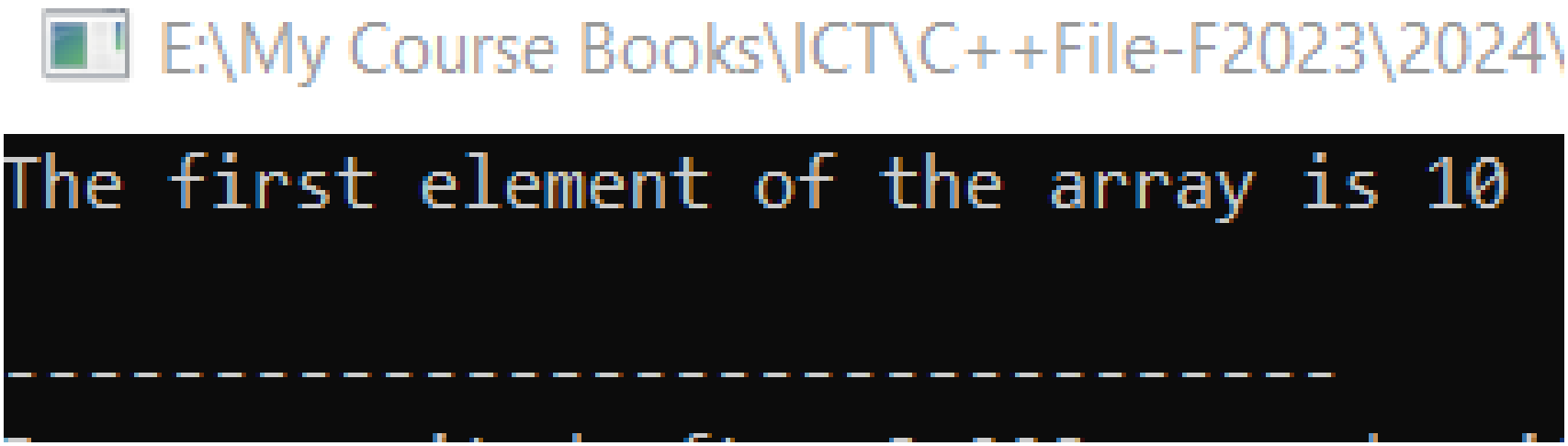
- the indirection operator being used with ptr
- That means the statement is not affecting ptr , but the item that ptr points to
- This statement assigns 100 to the item ptr points to, which is the x variable
- After this statement executes, 100 will be stored in the x variable

Arrays and Pointers

- An array name, without brackets and a subscript → represents the starting address of the array
- This means that an array name is really a pointer

EX-4

```
int numbers[] = {10, 20, 30, 40, 50};
cout << "The first element of the array is ";
cout << *numbers << endl;
```



You must try to run Program 9-6
Tony Gaddis

Emergency help System-Q1

- You are developing a new system for emergency service in Punjab (phone number 15)
- The main program asks the user to dial one digit code (enter 1 digit code) to request some kind of help
- The codes are : 1, 2, 3, 4, 5, 6, 7
- When the user dials a number, your main() function calls a recursive function that displays a message on the screen infinitely.
- Sample messages are: help, breathing difficulty, fire, vehicle accident, head injury, fainted patient, corona

Fortune Cookie game-Q2

- You have 10 fortune cookies and each cookie holds a message for you.
- Write a main() function that calls a function to generate a random number between 0 and 9. It returns the number to main()
- The main() function displays the message accordingly and asks if you want to play again or quit
- The player can play for not more than 5 turns and may quit before this.
- Design a fortune cookie game with interesting wishes for the player's fortune

Book Ref

- Ch-9